Benchmark Ollama pour 8 Go de RAM

Quel modèle local pour un détecteur de tendances arômes et parfums ?

Ce document n'est pas un classement mesuré. Je n'ai pas pu exécuter Ollama dans mon environnement : aucun chiffre de vitesse ou de précision ci-dessous n'a été mesuré par moi. Il contient (1) une présélection argumentée de modèles qui tiennent dans 8 Go, (2) un protocole de test adapté à ton cas d'usage, (3) un script prêt à lancer qui remplit le tableau de résultats en une dizaine de minutes. Les tailles de téléchargement sont des ordres de grandeur issus de ma mémoire des fiches Ollama : vérifie-les avec `ollama list`.

1. Contrainte : ce qui tient dans 8 Go

Les 8 Go sont partagés avec le système, le navigateur et ton éditeur. Il faut compter le modèle quantifié (Q4 par défaut) **plus** le cache de contexte. En pratique, vise un modèle de **moins de 4 Go** sur disque, soit environ 1 à 4 milliards de paramètres. Un modèle de 7-9 Md (Qwen 3.5 9B, environ 6,6 Go selon un test public) est trop gros : il fera swapper la machine ou sera très lent. Ferme les autres applications pendant les tests.

Ta tâche (classer, extraire, résumer, renvoyer du JSON) est exactement ce que les petits modèles font correctement. Ce qu'ils font mal : savoir quelles tendances existent. Ils doivent traiter des signaux que *tu* fournis, jamais les inventer.

2. Modèles présélectionnés

Modèle (tag Ollama)	Taille approx.	Pourquoi le tester	Point de vigilance
qwen3.5:4b	~3 Go (à vérifier)	Famille la plus multilingue (201 langues annoncées), bonne tenue sur les consignes et le JSON. Mon favori sur le papier.	Récent : vérifie que ta version d'Ollama le supporte. Désactiver le mode réflexion pour la vitesse.
qwen3:4b	~2,5 Go	Multilingue, mode réflexion activable/désactivable, très répandu.	Le mode réflexion ralentit fortement : le script le coupe.
gemma3:4b	~3,3 Go	Bon en rédaction/résumé, multilingue, accepte aussi les images.	Le plus lourd de la liste : surveille la RAM.
llama3.2:3b	~2 Go	Rapide, souvent cité comme meilleur généraliste à ce niveau.	Plus faible hors anglais ; extraction FR/ES/ID à vérifier.
phi4-mini	~2,5 Go	Bon raisonnement pour sa taille (3,8 Md).	Orienté anglais ; moins à l'aise en multilingue.
qwen3.5:2b / **qwen3:1.7b**	~1,5-2 Go	Plan B si la machine est lente : classification simple en gros volume.	Résumés et extraction fine moins fiables.
bge-m3 (embedding)	~1,2 Go	Pas un LLM : vectorise les textes en plusieurs langues pour regrouper les signaux par thème et suivre leur croissance.	Se teste à part (regroupement), pas avec le script ci-dessous.

Sources de la présélection : comparatifs publics de 2026 (Qwen 3.5 : 201 langues ; Qwen3 : 119 langues, fort en consignes ; recommandations 8 Go : llama3.2 3B et phi4-mini). Ce sont des blogs, pas des benchmarks reproductibles : d'où le protocole ci-dessous.

3. Protocole de test

On mesure ce dont ton projet a besoin, pas des scores génériques.

Critère	Comment on le mesure	Pourquoi
JSON valide (%)	La réponse se parse-t-elle ? (10 textes)	Ton pipeline plante si le JSON est cassé.
Famille correcte (%)	goût / parfum / autre vs étiquette attendue	Le tri goût-parfum est la base du détecteur.
Ingrédient extrait (%)	L'ingrédient attendu apparaît dans la réponse	C'est le signal faible que tu cherches.
Vitesse (tokens/s)	eval_count / eval_duration renvoyés par Ollama	Combien de posts traitables par heure.

Critère	Comment on le mesure	Pourquoi
Latence moyenne (s)	Temps total par texte, modèle déjà chargé	Confort de la démo.
RAM utilisée (Go)	`ollama ps` pendant le test	Doit rester sous ~6 Go pour ne pas swapper.

Jeu de test : 10 textes fictifs courts en français, anglais et espagnol (yuzu à Séoul, ube à Manille, oud en Arabie saoudite, un intrus sans rapport). Température 0 pour des résultats reproductibles. Dix textes ne prouvent rien statistiquement : le but est d'écarter les modèles inutilisables. Pour départager les deux ou trois finalistes, remplace le jeu par 30-50 textes réalistes que tu étiquettes toi-même.

Lancer : `ollama pull qwen3.5:4b llama3.2:3b gemma3:4b` (un pull par modèle), puis `python bench_ollama.py qwen3.5:4b qwen3:4b gemma3:4b llama3.2:3b`. Le script écrit `resultats.csv`.

4. Tableau de résultats à remplir

Modèle	JSON %	Famille %	Ingr. %	Tok/s	Latence s	RAM Go	Verdict
qwen3.5:4b							
qwen3:4b							
gemma3:4b							
llama3.2:3b							
phi4-mini							
qwen3.5:2b							

Règle de décision : garde le modèle qui dépasse ~90 % de JSON valide, la meilleure précision de famille, et reste sous 6 Go de RAM. À précision égale, prends le plus rapide. Si deux modèles se valent, le plus multilingue l'emporte : le goût et le parfum varient fortement selon les pays.

5. À retenir pour le dossier

• Le modèle ne détecte pas la tendance : il structure les signaux (famille, ingrédient, lieu). La détection vient du **comptage dans le temps et par pays** sur ces signaux structurés (tableur ou pandas).
• Un humain valide avant toute décision (lancement, brief client), comme le guide le demande.
• Mentionne dans ton dossier que le test a été fait sur machine 8 Go, avec un jeu fictif : c'est honnête et défendable.
• Argument confidentialité : un modèle local garde les données sur la machine.

Annexe : script bench_ollama.py

Même contenu que le fichier fourni séparément. Aucune dépendance externe (Python 3 uniquement).

```
#!/usr/bin/env python3
"""Mini benchmark Ollama pour le détecteur de tendances (machine 8 Go).
Usage : python bench_ollama.py qwen3.5:4b llama3.2:3b gemma3:4b
Prérequis : Ollama lancé (ollama serve), modèles déjà téléchargés (ollama pull ...).
Sortie : résultats.csv + tableau dans le terminal. Aucune dépendance externe.
"""
import json, sys, time, csv, urllib.request

URL = "http://localhost:11434/api/chat"

# Jeu de test FICTIF : (texte, famille attendue, ingrédient/note attendu(e) en minuscules)
DATA = [
 ("Everyone in Seoul is making yuzu honey iced tea this summer", "gout", "yuzu"),
 ("Les créateurs TikTok adorent le piment fermenté sur les glaces à la vanille", "gout", "piment"),
 ("Nueva ola de mocktails con jengibre y hibisco en Ciudad de México", "gout", "hibisco"),
 ("Le nouveau parfum d'ambiance à l'oud et à la figue cartonne en Arabie saoudite", "parfum", "oud"),
 ("Fragrance layering with vetiver and pink pepper is trending among Gen Z in London", "parfum", "vétiver|vetiver"),
 ("Les eaux de toilette à la tubéreuse reviennent dans les parfumeries de Paris", "parfum", "tubéreuse|tubereuse"),
 ("Ube lattes are popping up on café menus across Manila and Los Angeles", "gout", "ube"),
 ("Savon artisanal au néroli et à la fleur d'oranger : forte hausse des recherches en Italie", "parfum", "néroli|neroli|fleur d'oranger"
 ("Le prix de l'essence a encore augmenté ce week-end en Allemagne", "autre", ""),
 ("Matcha au sel de mer et à la mangue : nouvelle boisson virale à Tokyo", "gout", "matcha|mangue"),
]

SYSTEM = ("Tu es un analyste de tendances arômes et parfums. Pour chaque texte, réponds "
          "UNIQUEMENT par un JSON : {\"famille\": \"gout|parfum|autre\", "
          "\"ingredient\": \"<ingrédient principal ou vide>\", "
          "\"pays_ou_ville\": \"<lieu cité ou vide>\", "
          "\"resume\": \"<une phrase en français>\"}")

def call(model, text):
    body = {"model": model, "stream": False, "format": "json",
            "options": {"temperature": 0},
            "messages": [{"role": "system", "content": SYSTEM},
                         {"role": "user", "content": text}]}
    if model.startswith("qwen3"):          # désactive le mode "thinking" (plus rapide)
        body["think"] = False
    req = urllib.request.Request(URL, json.dumps(body).encode(),
                                 {"Content-Type": "application/json"})
    t0 = time.time()
    with urllib.request.urlopen(req, timeout=300) as r:
        out = json.load(r)
    return out, time.time() - t0

def run(model):
    ok_json = ok_fam = ok_ing = 0
    speeds, latencies = [], []
    call(model, "warmup")                  # chargement du modèle (non compté)
    for text, fam, ing in DATA:
        out, dt = call(model, text)
        latencies.append(dt)
        if out.get("eval_duration"):
            speeds.append(out["eval_count"] / (out["eval_duration"] / 1e9))
        try:
            j = json.loads(out["message"]["content"]); ok_json += 1
        except Exception:
            continue
        ok_fam += str(j.get("famille", "")).lower() == fam
        pats = [p for p in ing.split("|") if p]
        ok_ing += (not pats and not j.get("ingredient")) or \
                  any(p in str(j.get("ingredient", "")).lower() for p in pats)
    n = len(DATA)
    return {"modele": model, "json_valide_%": 100*ok_json//n, "famille_%": 100*ok_fam//n,
            "ingredient_%": 100*ok_ing//n,
            "tokens_par_s": round(sum(speeds)/len(speeds), 1) if speeds else 0,
            "latence_moy_s": round(sum(latencies)/len(latencies), 1)}

if __name__ == "__main__":
    models = sys.argv[1:] or ["qwen3.5:4b", "llama3.2:3b", "gemma3:4b"]
    rows = []
    for m in models:
        print(f"-> test de {m} ...", flush=True)
        try: rows.append(run(m))
        except Exception as e: print("   échec :", e)
    if rows:
        with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=rows[0].keys()); w.writeheader(); w.writerows(rows)
        for r in rows: print(r)
        print("Écrit : resultats.csv  (note aussi la RAM via 'ollama ps')")
```